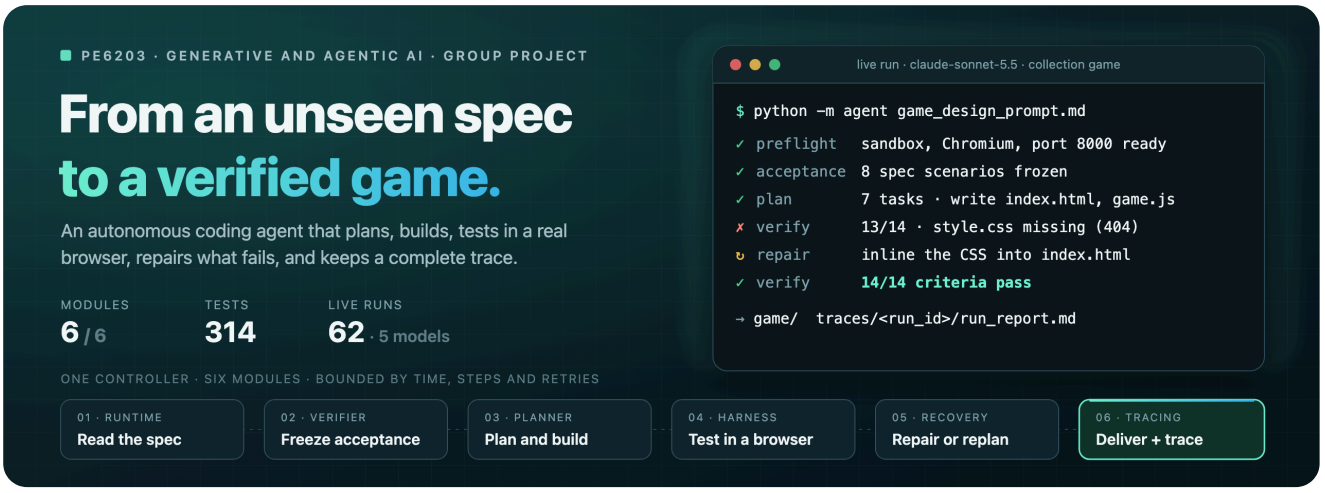
PE6203 · GENERATIVE AND AGENTIC AI · GROUP PROJECT 1

A Coding Agent That Builds a Game

Project report: architecture, planning and verification, failure recovery, evaluation, limitations and next steps



Member	Name	Module owned
1	ZHOU YU	Controller, CLI, configuration, team contract, integration
2	JIN ZIYI	Planner: model client, prompts, plan and decision parsing
3	DAI MINFEI	Tool runtime: files, commands, sandbox, processes, dependencies
4	ZHOU SIHAN	Verifier: acceptance criteria and the Playwright game harness
5	LIN SIYUAN	Failure recovery policy and offline evaluation
6	LI JIAKUN	Tracing, run report and delivery audit

Repository: github.com/shirley830/6203-game-coding-agent · Entry point: `python -m agent game_design_prompt.md`

Instructor: Prof. Bo An · Teaching assistant: Feng Lang · October 2026

1 Summary

We built an autonomous coding agent that starts from a single, previously unseen `game_design_prompt.md`, writes a complete game into `game/`, and leaves a full execution record in `traces/`. The agent is started with one documented command and needs no human help afterwards. Its distinguishing design choice is that **success is never declared by the language model**: a deterministic controller grants success only when every mandatory acceptance criterion passes on the current code, with evidence, and those criteria are derived from the specification *before* any code exists and checked by actually playing the game in a headless browser.

9 / 10	62	314	6
runs fully accepted in the latest Kimi K3 retest (course model, 6 specifications)	autonomous runs with five real models	automated tests, all passing	modules behind one controller and one contract

1 · Summary
2 · System architecture and rationale
3 · How the agent plans
4 · How the agent verifies its work
5 · How the agent recovers from failure
6 · Runtime safety and tracing
7 · Evaluation
8 · Current limitations
9 · Next planned improvement
10 · Use of AI assistance
11 · Member contributions
Appendix · Running the agent

2 System architecture and rationale

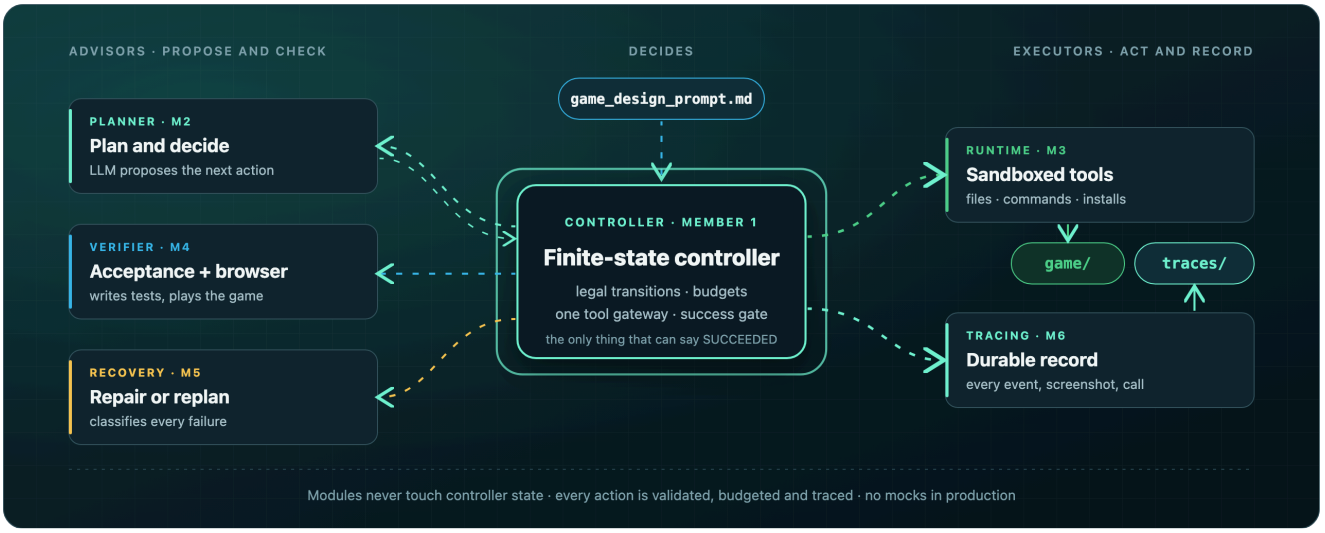


Figure 1. One finite-state controller decides. The planner, verifier and recovery modules propose and check; the sandboxed runtime and tracing act and record.

The agent is a **single sequential controller** (Member 1) with five injected modules. Modules receive an immutable view of the run and return a typed proposal or observation; they cannot change controller

state, call each other, or run tools themselves. Every file write, command and dependency installation passes through one tool gateway in the controller, which validates its scope, charges it to the budgets, records it in the trace and only then hands it to the runtime. All modules are wired by one production factory (`agent/integrations.py`); test doubles exist only under `tests/`.

Module	Contract	Responsibility
Controller (M1)	state machine, budgets	Phases, legal transitions, tool gateway, stopping conditions, the success gate
Planner (M2)	`create_plan`, `next_decision`	Plan from frozen criteria; propose exactly one next action per call
Runtime (M3)	`execute`	Files, commands, installs inside an OS sandbox; process-group cleanup; workspace revisions
Verifier (M4)	`prepare_acceptance`, `next_check`	Spec-derived acceptance; read-only checks; browser play-testing; evidence
Recovery (M5)	`recommend_recovery`	Classify each failure; recommend RETRY, REPAIR, REPLAN or ABORT
Tracing (M6)	`open_run`, `append_event`, `finalize`	Durable events, checkpoints, evidence, run report, delivery check

Design rationale and trade-offs

- **One controller instead of a multi-agent system.** A single owner of state makes every decision traceable and every budget enforceable, and it removes coordination failures between agents. The cost is less parallelism; for one game in a bounded time slot, predictability mattered more than throughput.
- **The model proposes, the controller decides.** Language models are good at generating code and poor at judging their own completion. Success therefore requires a verification report that matches the current specification digest, workspace revision and acceptance-suite version, with evidence for every mandatory criterion.
- **Typed contracts first.** The team agreed a written interface contract (`docs/INTERFACES.md`) before implementation, so six members could build modules in parallel and integrate them without changing each other's code.
- **Browser games by default.** A self-contained HTML5 canvas game needs no build step or installation and can be verified by a real browser. The agent builds Python terminal or pygame games only when the specification asks for them, as the course allows both.
- **Provider independence.** The model is reached through LiteLLM, so changing the course model from GLM-5.3 to Kimi K3 in October 2026 required only configuration and a reasoning cap (Sections 7.3 and 7.4).

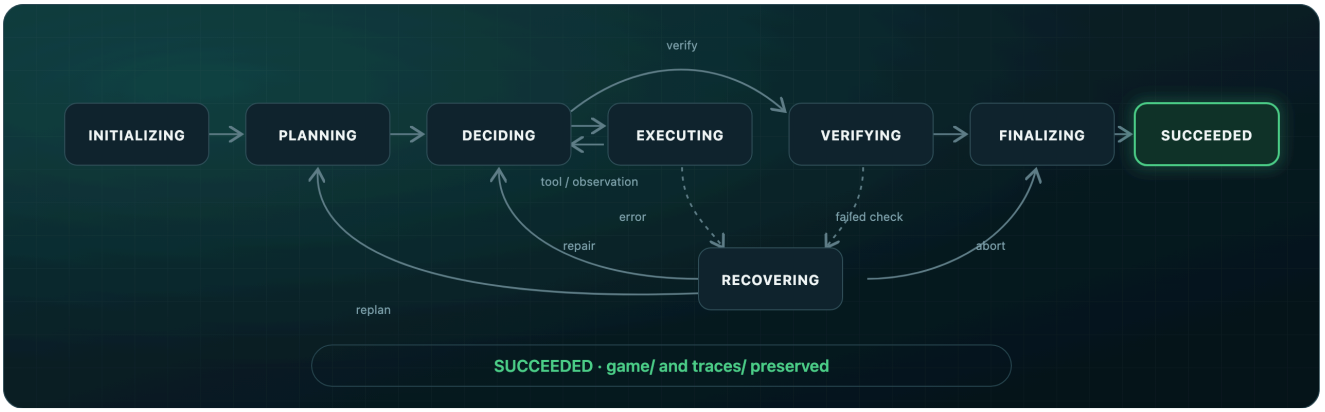


Figure 2. The run state machine. Errors, failed checks and uncertain outcomes go to RECOVERING, which repairs (back to DECIDING), replans (back to PLANNING) or aborts. Budget exhaustion or cancellation finalizes from any phase; finalization always preserves `game/` and `traces/`.

3 How the agent plans

Plan. After the acceptance suite is frozen (Section 4), the planner asks the model for 3-8 concrete tasks. Each task must reference acceptance criteria; the plan must cover every mandatory criterion and have an acyclic dependency graph, otherwise it is rejected and regenerated. The plan prompt treats the specification as data that cannot override system policy.

Decide, one action at a time. In the build loop the planner returns exactly one decision per call: a tool action (write, patch, read, inspect, run, install), a verification request with a launch command, or a plan revision. Because the controller exposes only the latest result, the planner keeps its own **working memory**: the files in `game/` together with their **current text** (kept up to date across writes, reads and accepted patches), the last 25 actions and failures, guard notes, and facts from the start-up self-check (for example "node is not installed"). Showing the current code in every decision removed a costly pattern seen with Kimi K3: after a failed check the model re-read files one at a time, losing about 200 s of a 400 s run.

Write once, then patch. Each file is written in full once; later fixes are unified-diff patches that change only the affected lines, because model output is the slowest part of a run. Models often miscount hunk headers, so the runtime places a rejected hunk by its content (it must match exactly once); otherwise nothing is written. Step-by-step decisions use a lower reasoning cap (3 000 tokens) than planning and acceptance (8 000).

Guards. Some rules are enforced in code rather than trusted to the model:

- A verification request is refused if the launch command refers to files that do not exist; `game/README.md` must contain the exact launch command and is written deterministically if missing.
- After eight consecutive edits without verification, the planner requests verification itself. Live runs showed a model rewriting one file 35-45 times without fresh feedback until the time budget ran out.
- The planner owns action IDs (models reuse example IDs) and repairs unambiguous format slips, such as `{"decision": "read_file"}` instead of a tool action of kind `read_file`. Calls use JSON mode.
- **Specification fidelity.** The prompt forbids adding controls, shortcuts or behaviour outside the specification to make a check pass, and requires fixing the real defect instead (Section 4.4).
- **Presentation (Member 2).** The first complete version must already be presentable (title, status bar, styled controls, clear win and loss messages), with all styling inline in `index.html` and every

referenced file present. Visual polish never delays verification, because a passing verification ends the run.

4 How the agent verifies its work

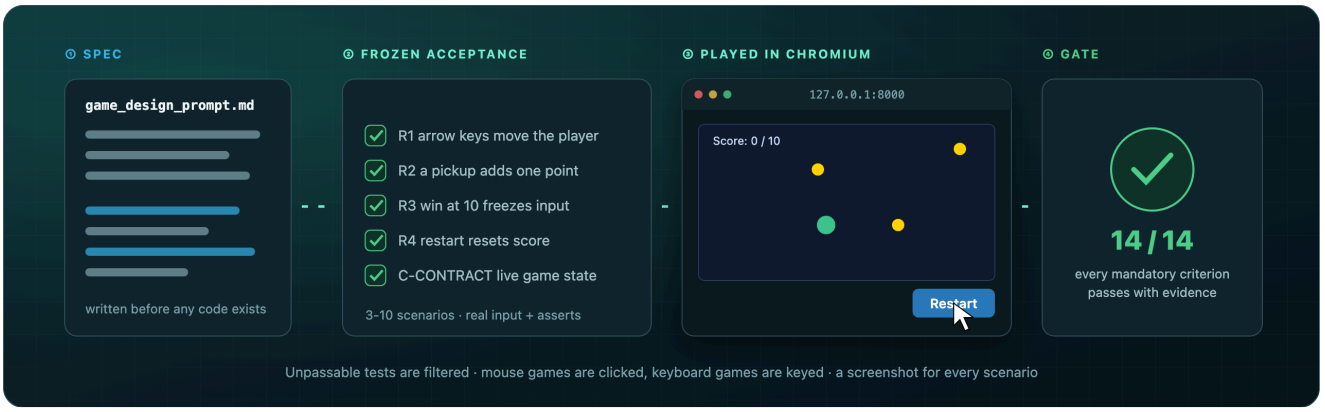


Figure 3. Verification: the specification becomes frozen executable scenarios before any code exists; the harness plays the game in headless Chromium; failures return to the planner with the failing step and the game state.

4.1 Acceptance is frozen before planning

Immediately after reading the specification, the verifier asks the model, in the role of an independent tester, for a **state contract** and **3-10 executable scenarios**. The contract requires the game to expose its real live state as `window.__gameState` and a few setup hooks as `window.__gameTest` (for example placing a collectible next to the player). A scenario is a list of real inputs (key presses, held keys, clicks on DOM elements or grid cells) and JavaScript assertions over the state. These criteria are frozen with the generic launch, functional, interaction and technical criteria; neither planning nor repair can change them.

Model-written tests can be wrong, so each requirement is validated on its own: invalid steps, undeclared hooks, canvas clicks in a specification that explicitly requires the keyboard, or assertions that use undefined names (Member 4) drop that requirement only. The tester prompt also requires that a setup must not end the game before the expectation unless the ending is what is tested. If the reply is unusable, the verifier asks once more with the field errors; if fewer than three requirements survive, the generic criteria decide the run and an informational `C-ACCEPTANCE-INCOMPLETE` result keeps the gap visible. Such a run is never reported as full success (Section 4.3).

Disputed checks. A check can also look valid and still contradict the specification. In a Kimi K3 retest a minesweeper check placed the mines so that the first click flood-revealed the whole board and won, but expected the game to be still playing; the correct game failed it twice and the run spent its budget "repairing" it. A check is now disputed (`test_conflict`) when the planner returns `dispute_check` with the specification rule as evidence, or when the same check fails in exactly the same way before and after a game change. The acceptance model re-examines that one check: *keep* sends the planner back to fixing the game; *replace* substitutes a check of the same rule (same id and category, at least one expectation), and the unchanged game is verified again. Each check is re-examined at most once and at most two per run, the old and new versions are kept in the event log, and the original acceptance and specification are never edited.

4.2 The game is actually played

The harness starts the documented launch command, loads the page in headless Chromium and records JavaScript errors. It checks the contract on the live object, then runs every scenario on a freshly loaded page, saving a screenshot per scenario. A failing scenario reports the step that failed and the state after it, for example `step 5 {"expect": "s.status === 'lost'"} … state after: {...}`, which is exactly what the planner needs to repair the game. A generic interaction probe (keys, canvas points, board cells, buttons) and checks for a play surface complement the scenarios. Terminal and pygame games are verified in a process mode: start without a traceback, keep running, react to typed moves (pygame input can only be checked statically and is labelled as such).

4.3 The success gate

The controller computes the outcome; a verifier cannot simply claim success. A report passes only if its specification digest, workspace revision and acceptance-suite version are current, every criterion has a result, every mandatory result is PASS with non-empty evidence, at least one verification tool operation succeeded in this cycle, a launch command exists, and the trace is durably preserved.

The final output separates three **result levels** (Member 1), also stored in `summary.json` and `run_report.md`: launch (the page loaded), specific rules (how many spec-derived checks passed, and which were re-derived after a dispute) and delivery (`game/` with README and launch command). Only when all three are complete is the run shown as *fully accepted*; a run that passed the generic checks alone is shown as *acceptance incomplete*. The levels do not change the outcome or the exit code.

4.4 Why this matters: evidence from live runs

Observation	Consequence
With generic checks only, all six first-round runs "succeeded", but manual review found specification violations in three of them (for example, walking into the enemy did not lose).	Spec-derived scenarios now catch these: the same "enemy contact loses" rule failed in every cycle on the faulty game, and GLM-5.3 fixed it after one failed cycle.
When the harness could only press keys, the agent added keyboard controls and click-anywhere behaviour to a mouse-only memory game to make the probe pass.	Mouse-aware scenarios and the fidelity rule removed this: later runs passed by real clicks with no extra controls.
A memory-card specification that says "turn over two cards" but never says "mouse" lost all its scenarios to the mouse filter, and the run "succeeded" on generic checks with 0 specific rules.	The filter now drops clicks only in explicitly keyboard-only specifications; the same specification then passed with 9/9 and 5/5 specific rules, and the result levels show such a gap instead of full success.
A minesweeper check whose setup made the first click win led the agent to cap the flood reveal at three cells to satisfy it (a Member 5 retest).	With disputed checks the replacement placed the mines so that the flood stops before a win; the unchanged, standard flood reveal then passed 9/9.

5 How the agent recovers from failure

Every failure is recorded with its phase and category and goes to the recovery module, which classifies it and recommends one of five decisions. The controller enforces the decision and all limits; modules cannot

reset counters.

Failure class	Decision	Reasoning
Transient model error (timeout, connection, rate limit, 5xx)	RETRY	Proposal-only calls are safe to repeat; waits 5 s, 15 s, 45 s, 60 s before each retry, within the deadline
Malformed model output	REPAIR / REPLAN	Ask again with the parse error visible; an invalid plan is regenerated
Command failed, file missing	REPAIR	Inspect output and fix; never replay the unchanged command
Timeout or unknown side effects	REPAIR	**Never retried blindly**; earlier verification is invalidated and the workspace re-inspected
Failed verification	REPAIR, periodically REPLAN	The failing steps go to the planner; repeated failure rebuilds the plan
Broken test script (`test_script_error` on a mandatory check)	ABORT	The test, not the game, is at fault; the game is never edited to satisfy it
Disputed check (`test_conflict`)	REVISE_ACCEPTANCE	Re-examine that one check against the specification, then verify the unchanged game (Section 4.1)
Permission, sandbox or credential error; API key out of credit; process cleanup failure	ABORT	Retrying cannot help; preserve artifacts and finalize honestly with a clear reason

Stopping conditions bound every run: total time (including a finalization reserve), steps, commands, retries per operation, global recovery attempts, and repeated no-progress cycles. Any stop still finalizes the trace and keeps `game/`. Before every verification the controller also snapshots the game source into the trace; if a run ends without success, it restores the version that passed the most mandatory criteria and records the decision. The outcome stays FAILED or EXHAUSTED: a better partial version never counts as success. Two later additions (Member 5): the same failure on an unchanged workspace triggers a replan after three occurrences and an abort after five, and a verifier fault (a broken test script on a mandatory criterion) aborts instead of "repairing" a game that is not at fault. In live runs this machinery handled real provider timeouts, malformed JSON (including empty `{}` replies from GLM-5.3, which the planner now absorbs without spending recovery attempts), runtime errors in generated code, and missing files.

6 Runtime safety and tracing

Runtime (Member 3). Commands run as argument vectors (no shell) inside an OS sandbox (macOS `sandbox-exec`, Linux `bwrap`) that allows writes only to `game/` and the run's private storage, so generated code cannot modify the agent, the specification or the traces. Each command runs in its own process group, which is terminated and confirmed gone after every command, timeout or cancellation. Dependencies install into an isolated environment. Credentials stay in the agent process: commands get a filtered environment and cannot read `agent.env`. A start-up self-check stops the run before `game/` exists if Chromium, the sandbox, a free port or Python is missing; `python -m agent.preflight --probe-model`

additionally makes one model call that must return valid JSON, so a wrong key, endpoint or reasoning setting is caught before the run.

Tracing (Member 6). Every controller event is appended to `events.jsonl` and is durable before the controller continues. The trace also holds checkpoints, every verification report, screenshots with a SHA-256 index, model usage per call (role, tokens, latency) and `run_report.md`, a readable timeline with each action's rationale, the frozen criteria, the plans and every verification cycle. `finalize` re-reads what it wrote before reporting the delivery as preserved, and `python -m agent.delivery` audits a finished trace, including `run_report.md` and every runtime output an action referenced against its archived copy and hash (Member 6).

7 Evaluation

7.1 Automated tests

314 tests cover the controller, configuration and model settings, planner, runtime and sandbox, verifier and harness (including real Playwright runs against fixture pages), recovery and evaluation, tracing and delivery audit, integration (including an offline end-to-end rehearsal with a scripted model and the real sandbox, browser, recovery and tracing), the Member 5 hand-off, and the acceptance signals added after the Kimi K3 retests: result levels, broken test scripts, disputed checks and their bounded re-derivation. A guard test ensures that the agent package contains no game code.

7.2 Live runs with real models

We ran the agent autonomously 62 times with five models: 48 runs on nine specifications during development (collection, dodge, memory cards, maze, terminal hangman, light switches, turn-based keys, grid snake and minesweeper) and 14 Kimi K3 runs on six specifications (Member 5's four, which include our minesweeper, and two never seen before), changing the agent between rounds based on what the traces showed.

Model	Runs	Succeeded	Observations
Kimi K3 (course model)	23	18	Three rehearsal rounds, then two retests of the shared acceptance fixes with the 7-minute limit; 9 of 10 runs fully accepted in the last one (Section 7.4)
GLM-5.3 (course model until October 2026)	16	14	Quality close to Claude Sonnet 5.5; more repair cycles on timing-heavy rules; one failure caused by a harness defect, fixed and re-run successfully
Claude Sonnet 5.5	11	11	Valid scenarios and spec-faithful games in every run
GPT-4.1-mini	8	5	Missed rule details; some generated scenarios could not be passed by a correct game
Qwen3-Coder	4	1	Frequent format errors and unpassable tests

Round 1 used generic checks only, so its successes are weaker evidence; later rounds used spec-derived scenarios. Each specification was run once per model and round, and the agent changed between rounds, so these figures indicate tendencies rather than rates. GLM-5.3: 9 runs during development and 7 in its final rehearsal (Section 7.3). Full data: `docs/testing/LIVE_MODEL_TESTS.md`.

7.3 The first course model: GLM-5.3

A probe before any run revealed a critical behaviour: with no reasoning setting, GLM-5.3 spent all 16 000 output tokens on reasoning and returned **no text** after 81 s, which would have stopped every planning call. Reasoning cannot be disabled for this model, but a reasoning cap makes it answer in about 5 s with valid JSON. The agent now caps reasoning at 8 000 tokens for GLM models. With that setting:

Specification	Outcome	Time	Repairs	Notes
Collection	succeeded	39 s	0	All 7 scenarios passed first time; reviewed against the spec
Terminal hangman	succeeded	17 s	0	Python, process mode; played by hand
Light switches	succeeded	115 s / 202 s	1 / 1	Two runs (before and after a harness fix)
Turn-based keys	succeeded	433 s	1	Caught and fixed "walking into the enemy does not lose"
Dodge	succeeded	453 s	5	Restart and timing rules took several cycles
Memory cards	succeeded	1151 s	8	Mouse only; used almost the whole 20-minute budget
Maze	failed, then succeeded	265 s / 235 s	20 / 6	First run hit a harness defect (function field in the state contract) and empty replies; both fixed

The probes and nine runs cost about US$1.7 in total, which also confirmed that the run fits within a presentation slot.

Final rehearsal. After the last changes, all seven specifications were run once more with GLM-5.3 through OpenRouter (a team member's key) and a 20-minute budget, with no human intervention:

Specification	Outcome	Time	Steps	Recoveries
Collection	succeeded	48 s	20	0
Memory cards	succeeded	45 s	22	0
Terminal hangman	succeeded	23 s	18	0
Light switches	succeeded	61 s	41	1
Turn-based keys	succeeded	106 s	30	0
Maze	succeeded	334 s	128	5
Dodge	exhausted	1180 s	166	8

The dodge run kept failing timing assertions in model-written scenarios (for example a score that had to be exactly 0 at the start of a check while the real-time loop was already adding points). At the deadline the controller restored the best assessed version (11 mandatory criteria passing) and reported the run as exhausted. The batch cost about US$0.5. Full data: `docs/testing/GLM_REPEAT_20261019.json`.

7.4 The course model: Kimi K3

In October 2026 the course switched to Kimi K3 through OpenRouter (`moonshotai/kimi-k3`). The agent needed only a configuration change: the model answered the probe with valid JSON in about one second. Kimi K3 bills reasoning as output at US$20 per million tokens, so it gets the same 8 000-token reasoning cap as GLM. We then rehearsed three specifications with the presentation limit of 7 minutes, changing the agent between rounds:

Round	Change before the round	Minesweeper	Grid snake	Turn-based keys	Cost
1	Model switch only	succeeded, 368 s	exhausted, 11/13	exhausted, 13/15	US$1.70
2	Repairs patch changed lines instead of rewriting files	exhausted, 13/14	succeeded, 331 s	succeeded, 77 s	US$0.65
3	Current file texts in every decision; lower reasoning cap for steps	succeeded, 165 s	succeeded, 299 s	succeeded, 338 s	US$0.64

The traces showed where the time went: waiting for the model took 330-370 s of each 400 s run in round 1, mostly in whole-file rewrites of 10 000-12 000 tokens (about 100 s each) and, after a failed check, in repeated `read_file` decisions. Round 3 removed both: no file was read again, each run needed 3 file writes and 1-3 patches, and model time fell to 141-290 s. Grid snake, a real-time game, had never passed within 7 minutes before round 2. Results vary with model speed from run to run, so these figures indicate the effect of each change rather than a success rate.

Retest of the shared acceptance fixes. Member 5 then ran four specifications of their own on the demo machine (detailed memory cards, minesweeper, detailed whack-a-mole, simple memory cards). Two problems were in the tests, not the games: the simple memory specification lost all its scenarios to the mouse filter, and the minesweeper and whack-a-mole checks allowed or demanded behaviour the specification forbids. After the fixes of Sections 4.1, 4.3 and 5 we ran the four specifications twice each and two unseen ones (Lights Out and tic-tac-toe against the computer) once, all with the 7-minute limit:

Specification	Run	Result	Specific rules	Time	Recoveries	Disputed check	Cost
Detailed memory cards	1	fully accepted	9/9	213 s	2	REQ-R7 replaced: clicked a row that does not exist	US$0.19
	2	fully accepted	7/7	304 s	2	REQ-R6 replaced: same coordinate error	US$0.34
Minesweeper	1	exhausted	7/10	400 s	1	none; one 23 000-token rewrite took 204 s	US$0.48
	2	fully accepted	9/9	280 s	3	REQ-R3 replaced: setup flood-revealed the board and won	US$0.36
Detailed whack-a-mole	1	fully accepted	10/10	230 s	1	none	US$0.28
	2	fully accepted	10/10	262 s	2	REQ-R3 replaced: expected no mole after the 300 ms respawn	US$0.21
Simple memory cards	1	fully accepted	9/9	87 s	0	none	US$0.22
	2	fully accepted	5/5	246 s	3	REQ-R1 kept: the game was at fault and was fixed	US$0.55
Lights Out (unseen)	1	fully accepted	8/8	102 s	0	none	US$0.17
Tic-tac-toe (unseen)	1	fully accepted	9/9	59 s	0	none	US$0.09

Nine of ten runs were fully accepted, in 218 s on average, for US$2.89 in total. Each of the five disputes was reviewed by hand: the four replacements test the same rule with the same expectations and fix only an impossible coordinate, setup or timing, and the one *keep* was right, because the game did not reveal the clicked card. The generated games use a standard flood reveal and count a missed mole in the last second, the two violations found in the earlier retest. Four further runs and the end of one more stopped at once because the team key reached its spending limit (HTTP 402); they are excluded and were re-run with a new key, and the agent now reports this case as "out of credit". A retest earlier the same day, before disputed checks existed, fully accepted three of the four Member 5 specifications; its minesweeper run spent the whole budget on the flood-reveal check.

7.5 Live run at the presentation

To be completed after the presentation. Specification received, outcome and reason, elapsed time, number of verification cycles and repairs, the most important failure and how the agent handled it, and where the evidence is in `traces/<run_id>/`.

8 Current limitations

- **Model-written tests can be wrong.** The scenarios come from the same model that writes the game. Filtering removes known bad patterns and a disputed check is re-examined once, but the re-examination is made by the same model, and a wrong test that a game happens to pass is not detected at all. Disputes therefore stay bounded and recorded, and every replacement must test the same rule.
- **Model speed decides many runs.** Waiting for the model is most of a run, and its speed varies: the same minesweeper specification took 368 s, ran out of time, and took 165 s in three Kimi K3 rounds, and in the last retest a single 23 000-token rewrite took 204 s of a 400 s run.
- **Real-time games are the weakest case.** Scenarios assert on wall-clock behaviour, so timers and continuous scoring needed up to eight repair cycles, and the dodge and snake games exhausted their budget in several rehearsals.
- **Non-browser games are verified less deeply.** Terminal games are played through standard input, but spec-derived scenarios apply to browser games only, and pygame input is checked statically.
- **Evaluation key and demo machine.** All runs used team keys for the same models and provider; the evaluation key is issued before the presentation and checked with `--probe-model`, and the presentation machine still needs a full rehearsal.
- **Small evaluation.** Fourteen specifications and at most three Kimi K3 runs per specification and agent version; results are indicative.
- **Platform.** The command sandbox requires macOS or Linux with bubblewrap; there is no resume after a hard interruption.

9 Next planned improvement

Deterministic time for real-time games. The two improvements planned earlier, keeping the best verified version and a model connectivity check, are now implemented (Sections 5 and 6). The remaining weak point is real-time games, whose scenarios currently race the game loop. The state contract will require a `pause` hook and a `step(ms)` hook that advances the simulation by a fixed amount, and the harness will run timing assertions on a paused game advanced step by step. Assertions such as "the score is still 0" or "an obstacle moves down" then become exact and repeatable instead of depending on frame timing, which targets the failure seen in the dodge rehearsal.

A smaller follow-up: **standard setup hooks** per game type (place an item next to the player, open a grid cell, set a score) to reduce unpassable model-written scenarios.

10 Use of AI assistance

The team used Codex (OpenAI) and Claude Code (Anthropic) as coding assistants during development. They were used to integrate the members' branches and diagnose and fix cross-module defects found during testing (for example, the data mismatch between the verifier and runtime, the interaction check that failed to detect player movement, and macOS process-group cleanup issues). Members designed and implemented their own modules (see Section 11) and reviewed the integration changes affecting their modules before they were merged into the main branch; subsequent changes were made by Member 1 with the same assistance and are recorded in the Git history.

At run time the agent itself calls only the course-designated model through LiteLLM. It does not invoke any existing coding agent, and it contains no pre-written game code: every game is generated during the run, and a test enforces that the agent package ships no game files.

11 Member contributions

Member	Contribution
1 · ZHOU YU	Team interface contract; finite-state controller, budgets and success gate; configuration and CLI entry point. • **Hardening:** start-up self-check, `agent.env` with credential isolation, retry backoff, and configurable run limits. • **Verification and model support:** spec-derived acceptance scenarios and the state contract, process-mode checks for terminal and pygame games, the GLM-5.3 reasoning cap, and empty-reply handling. • **Kimi K3:** switched the agent to the new course model and shortened repair cycles (patch-based repairs, current file texts in decisions, a lower reasoning cap for steps). • **Acceptance integration:** three result levels (launch, specific rules, delivery), the structured `test_script_error` signal, disputed checks with a bounded re-derivation, the out-of-credit stop, and provider-reported cost. • **Evaluation:** completed and analysed 32 live runs with four models over five rounds, three Kimi K3 rehearsal rounds, and two retests of the shared acceptance fixes. • **Integration:** merged the six members' branches into `main`, built the production dependency factory, fixed cross-module defects, and reviewed and merged the final handoff. • **Documentation:** README and illustrations, testing guides, and this report. • **Presentation:** prepared the presentation slides.
2 · JIN ZIYI	Planning module: model access, planning, and decision generation. • **Model integration:** implemented the LiteLLM model client. • **Structured output:** defined output schemas and implemented plan and decision parsing and validation. • **Prompts and validation:** developed planning and decision prompts, launch-command validation, and plan-coverage validation. • **Game quality:** added gameplay-completeness and presentation rules to the planner prompts. • **Testing:** implemented planner tests.
3 · DAI MINFEI	Tool runtime: workspace operations and command execution. • **File operations:** implemented workspace file operations, unified-diff patches, and workspace revisions. • **Hardening:** implemented the OS sandbox, process-group cleanup, and isolated dependency installation. • **Testing:** implemented runtime tests and prepared test reports.

4 · ZHOU SIHAN	Verification module: `GameVerifier` and acceptance criteria. • **Evidence and evaluation:** implemented the evidence ledger and criterion evaluation. • **Game testing:** implemented the Playwright game harness. • **Acceptance reliability:** detection of broken test scripts and incomplete acceptance, contradictory card-pair assertions, a pointer filter limited to keyboard-only specifications, and one regeneration of an unusable acceptance reply. • **Testing:** implemented verifier tests. • **Documentation:** prepared verification design notes.
5 · LIN SIYUAN	Recovery module: failure classification and recovery policy. • **Hardening:** model probe in the self-check, source snapshots, and best-version restore. • **Testing:** recovery tests and recovery-verifier integration tests. • **Evaluation:** offline evaluation specifications and the run-record report, and the final GLM-5.3 rehearsal. • **Handoff:** audited the historical run archive. • **No-progress control:** stops repeating unchanged failures and aborts on verifier faults, including the structured `test_script_error` signal. • **Kimi K3 retest:** ran four specifications on the demo machine, reviewed them by playing, and wrote the shared-fix handoff that assigned the work above.
6 · LI JIAKUN	Tracing and delivery: durable execution records and delivery auditing. • **Trace recording:** implemented the durable trace recorder and checkpoints. • **Evidence preservation:** implemented verification screenshot archiving with integrity checks. • **Delivery audit:** implemented delivery auditing, including missing-evidence detection, and extended it to the run report and archived runtime outputs. • **Testing:** implemented tracing tests.

A Appendix: running the agent

Step	Command
Install (once)	`pip install -e ".[test]"` then `python -m playwright install chromium-headless-shell`
Configure	`agent.env` next to the specification: `AGENT_MODEL=openrouter/moonshotai/kimi-k3`, `OPENROUTER_API_KEY=…`, `AGENT_MAX_RUNTIME=420` for the 10-minute slot
Check the machine	`python -m agent.preflight`, then `python -m agent.preflight --probe-model` (one billed call)
Run	`python -m agent game_design_prompt.md`
Start the game	the command in `game/README.md`, normally `cd game && python3 -m http.server 8000`
Inspect the run	`traces/<run_id>/run_report.md`; audit with `python -m agent.delivery traces/<run_id>`